

SC2001 Project 1

Integration of Merge Sort & Insertion Sort

Handwritten Python algorithms

Measured inputs from 1,000 to 10,000,000 integers

The hybrid algorithm

Stop splitting when the range has at most S items

```
hybrid(A, lo, hi, S)
  if hi-lo <= S:
    insertion(A, lo, hi)
    return
  mid = lo + (hi-lo)//2
  hybrid(A, lo, mid, S)
  hybrid(A, mid, hi, S)
  merge(A, lo, mid, hi)
```

Worked example: $S=3$

Input: [5, 2, 4, 1, 3, 6]

Leaves: [5, 2, 4] and [1, 3, 6]

Insertion: [2, 4, 5] and [1, 3, 6]

Merge: [1, 2, 3, 4, 5, 6]

$3 + 2 + 5 = 10$ key comparisons

Correctness, counting and fair timing

Correctness and counting

Insertion maintains a sorted prefix.
Merge emits the smallest remaining head.
Induction gives a sorted, stable result.

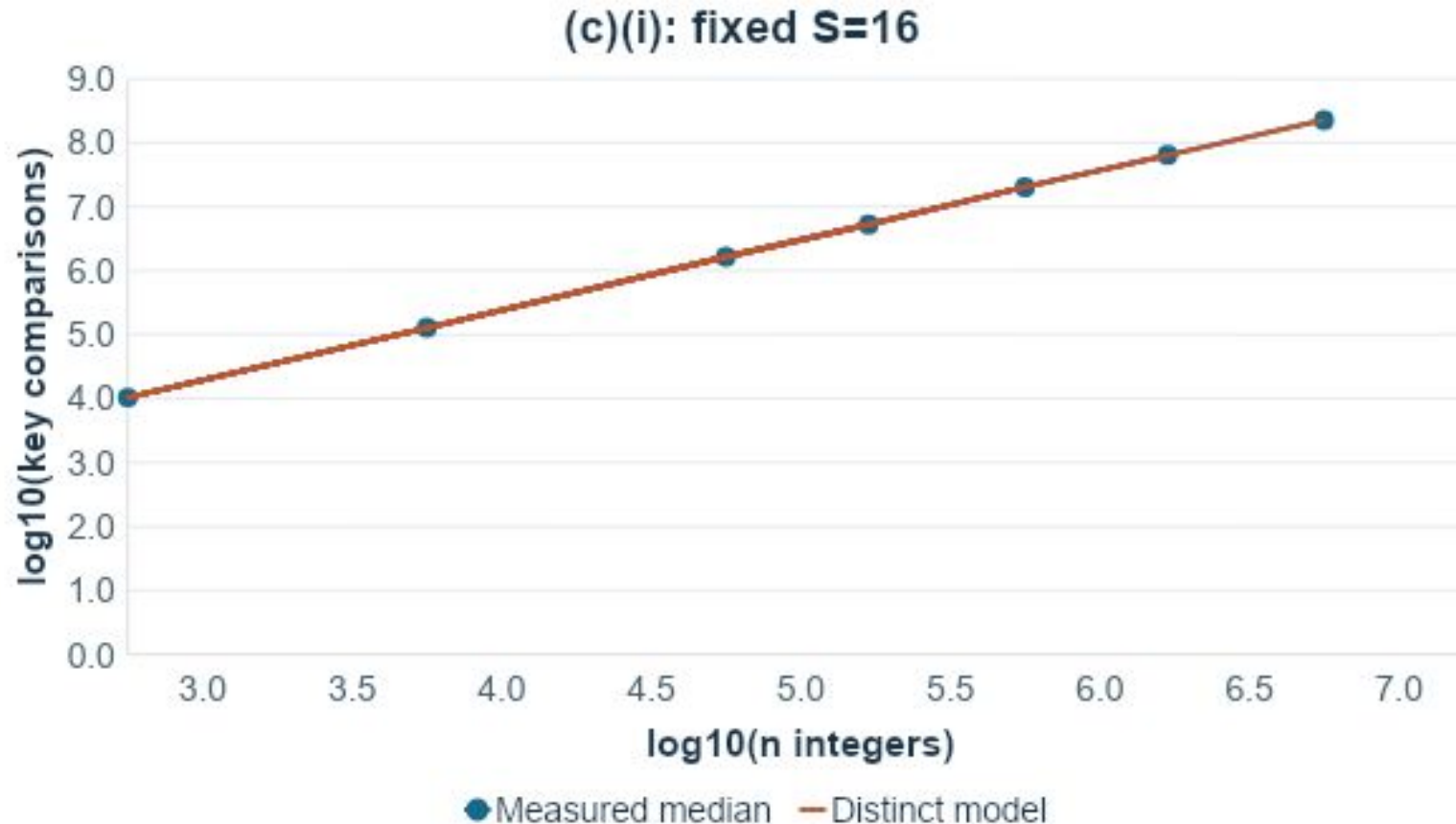
Count each evaluated data comparison:
merge $\leq$ and insertion $>$.
Include terminating false comparisons.

Experiment protocol

Uniform integers in $[1, 1,000,000]$.
Three seeds per case, paired inputs.
Separate counted and uncounted runs.
CPU clock: `time.process_time()`.

Exclude RNG, copies, buffer allocation,
validation and I/O. Run sequentially.
Verify order, length and exact histogram.

Fixed S retains $n \log n$ growth



Original

$T(n) = 2T(n/2) + \Theta(n)$

Time: $\Theta(n \log n)$

Buffer: $\Theta(n)$

Hybrid

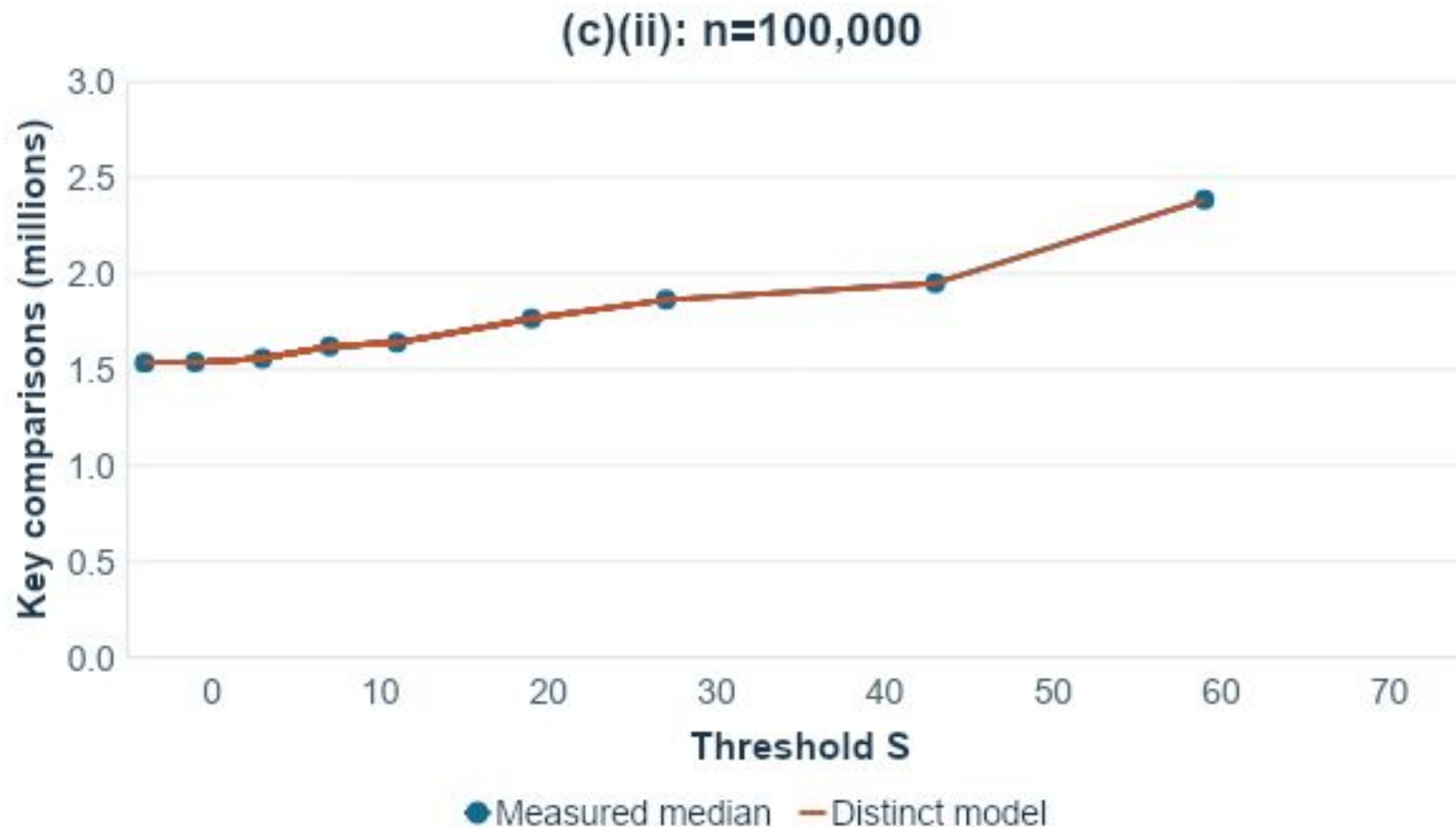
About $\log_2(n/S)$ merge levels

Random leaf work: $\Theta(nS)$

Fixed S : $\Theta(n \log n)$

Both axes show $\log_{10}$ values. Analytical counts use actual rounded splits with no fitted scale factor.

Increasing S changes the leaf work



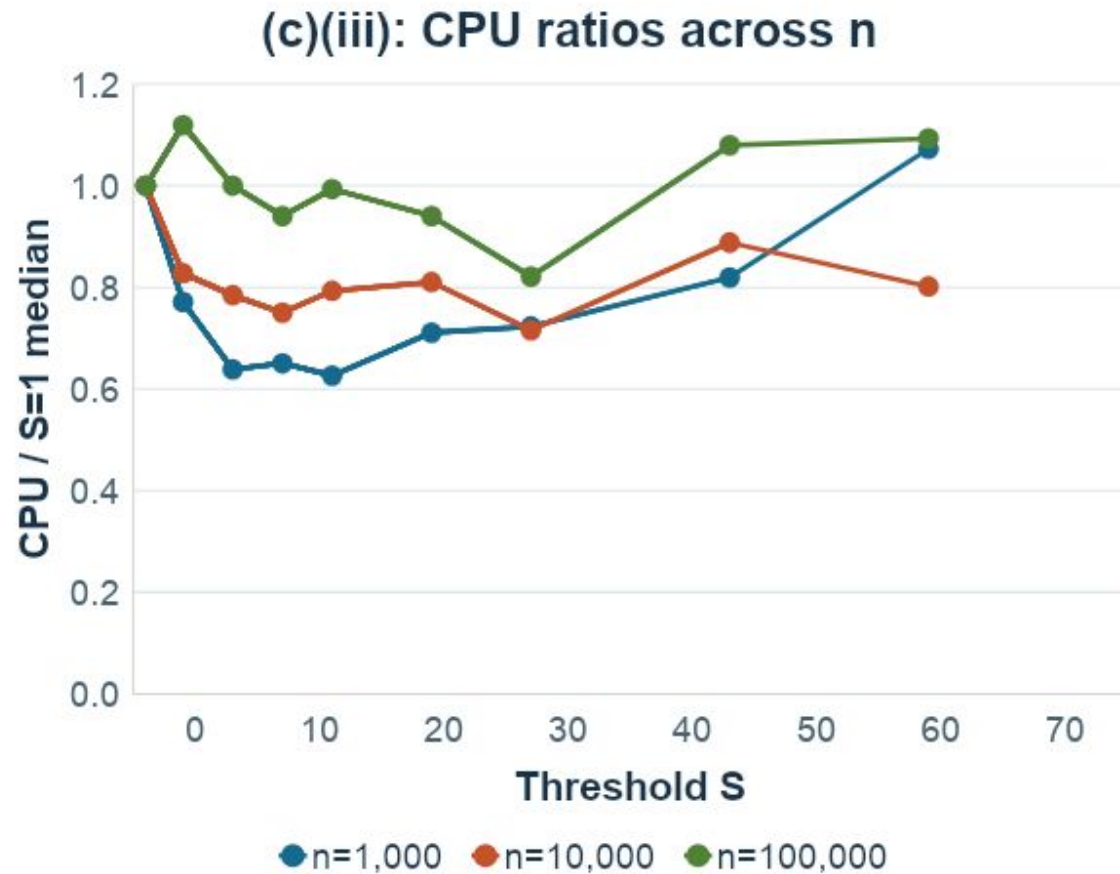
Fewer merge levels
Larger insertion leaves

Random insertion: $\Theta(S^2)$
per leaf
About n/S leaves
Total leaf work: $\Theta(nS)$

Some S values share the
same leaves.
Plateaus are expected.

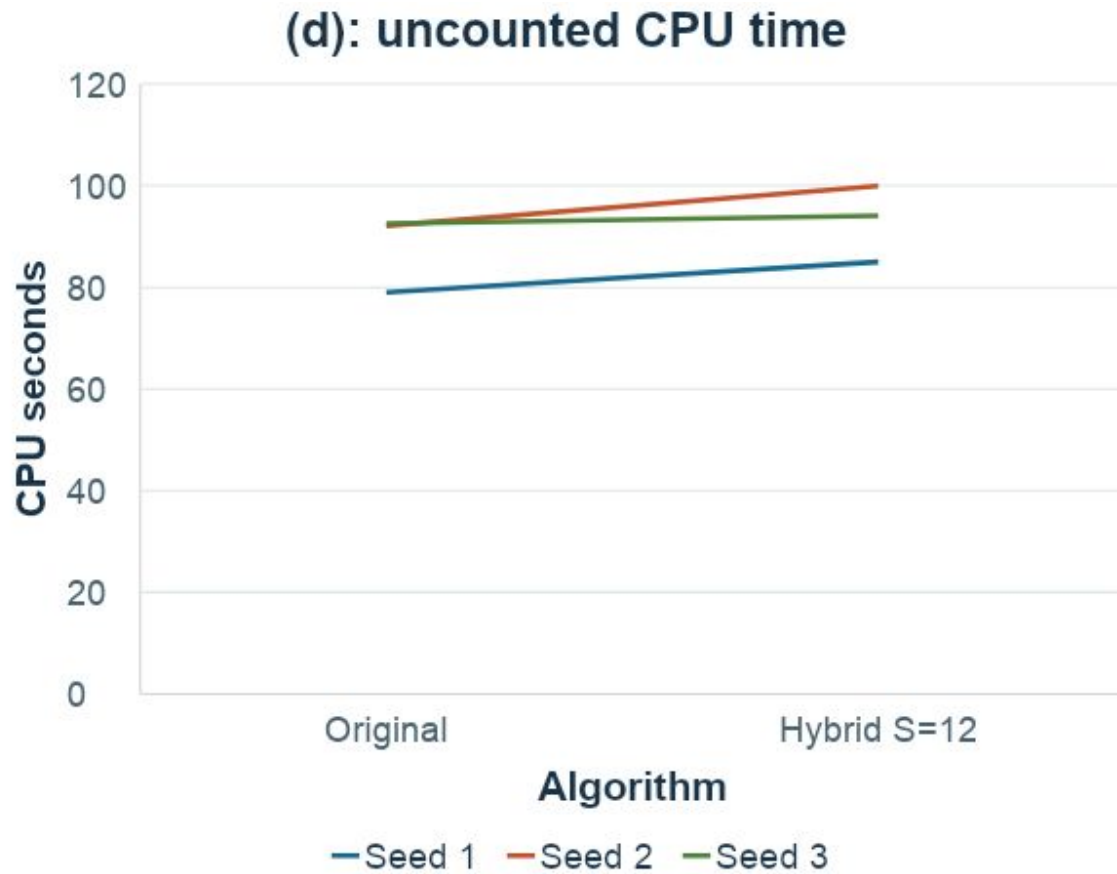
$S=1$ is included. Medians of three seeds. Full plots also show ranges and the worst-case bound.

CPU tuning selected S=12 by the tie-break rule



S=12, 32 tied at 2.78125 s median. The rule chooses smaller S. Three seeds per case.

Original and hybrid on 10 million integers



Median CPU: 92.094 s original, 94.078 s hybrid. Median paired speedup 0.929x.

Conclusions and questions

Hybrid was slower in 3 of 3 final CPU pairs.
Median paired speedup: 0.929x.

CPU time and key comparisons measure different costs.

S=12 follows a finite, noisy search and an explicit tie-break.

One interpreter, one machine, uniform integer inputs.

2 minutes of Q&A

Appendix: exact comparison models

Merge sort

Worst comparisons:

$$n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$$

Random distinct merge of lengths a,b:

$$a+b - a/(b+1) - b/(a+1)$$

Insertion expectation for m items:

$$m(m-1)/4 + m - H_m$$

$$H_m = 1 + 1/2 + \dots + 1/m$$

Hybrid regimes

Worst/average (random distinct):

$$\Theta(n \log(n/S) + nS), \text{ for } S \leq n$$

Fixed S: $\Theta(n \log n)$

$S = \log n$: $\Theta(n \log n)$

$S = n^\alpha$: $\Theta(n^{1+\alpha})$

$S \geq n$: insertion only

Buffer $O(n)$, recursion $O(\log n)$

Appendix: raw final measurements

Seed	Algorithm	S	CPU seconds	Key comparisons
100900100	original	1	78.968750	220,095,927
100900100	hybrid	12	85.000000	226,418,149
100900101	original	1	92.093750	220,099,689
100900101	hybrid	12	99.906250	226,424,963
100900102	original	1	92.515625	220,101,452
100900102	hybrid	12	94.078125	226,417,959

Each CPU value is from an uncounted run. Counts come from a separate run on identical unsorted data.

Appendix: different criteria, different winners

Input n	Lowest median CPU S	Fewest median comparisons S
1,000	16	1
10,000	32	1
100,000	32	1

Ties use the smaller S. These are per-size descriptive winners, before shortlist confirmation.